Chessapplicationdesign - Architecture Report

Application

The Application is a full-stack web application built with Next.js, Vite frontend and Python, FastAPI backend. The repository provides UI/code components. PRD specifies full features and architecture. (backend implemented)

Generated from:	GitHub Repository Analysis
Repository URL:	https://github.com/SanthoshDSR86/Chessapplicationdesign.git
PRD Analysis:	Included
Generated on:	2026-01-02 12:36:59
Analysis Scope:	68 files analyzed (full-stack analysis)

Architecture Pattern:	Client-Server Architecture, RESTful API Design
Application Type:	Web Application
Complexity Score:	9/10
Scalability Level:	Basic Scalability
Technology Maturity:	Modern

Table of Contents

1. Executive Summary
2. Architecture Goals & Scope
3. Architecture Context Diagram
4. Frontend Architecture
5. Backend Architecture
6. API Endpoints Documentation
7. System Interaction Flow (Sequence Diagram)
8. Component Interactions
9. Unified Architecture Diagram
10. Deployment Architecture
11. Security Model
12. Technology Stack Summary
13. Business Alignment
14. Recommendations & Next Steps
15. System Architecture Diagram

1. Executive Summary

This document presents a comprehensive analysis of the Application architecture, generated through automated analysis of the GitHub repository and associated documentation. Backend components detected and analyzed.

Key Findings:

- 1 API endpoints (repo + PRD) identified and documented
- 59 frontend components analyzed
- 3 backend technologies detected
- 2 programming languages in use
- 6 architectural recommendations provided

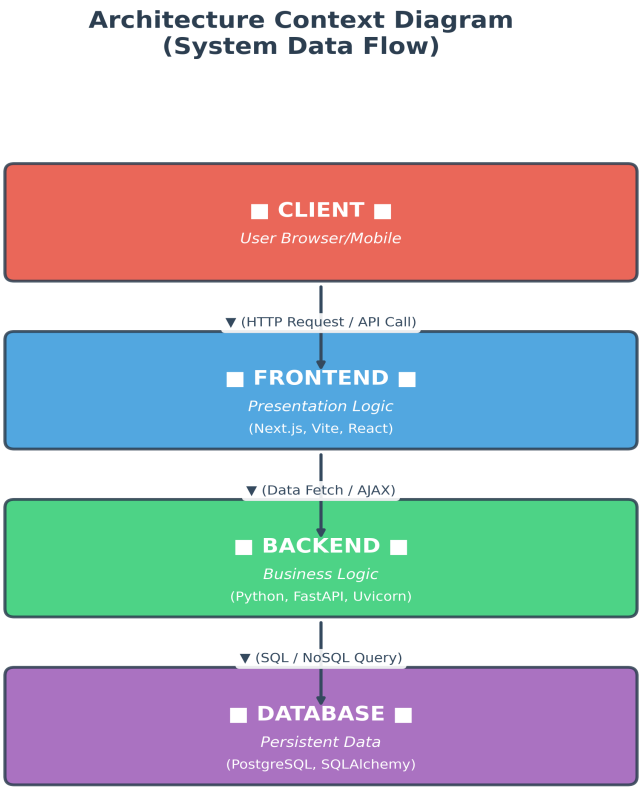
2. Architecture Goals & Scope

Architecture Goals:

- Deliver responsive and intuitive user interface
- Implement scalable backend services
- Ensure secure and maintainable code architecture
- Support efficient development and deployment workflows
- Enable cross-platform compatibility and performance

3. Architecture Context Diagram

The following diagram illustrates the high-level context of the system, showing how users interact with the frontend, which communicates with backend services, and how data flows to external systems and databases.



System Boundaries:

- Frontend: Next.js, Vite, React
- Backend: Python, FastAPI, Uvicorn
- Database: PostgreSQL, SQLAlchemy
- External Integrations: Authentication, APIs, Cloud Services

4. Frontend Architecture

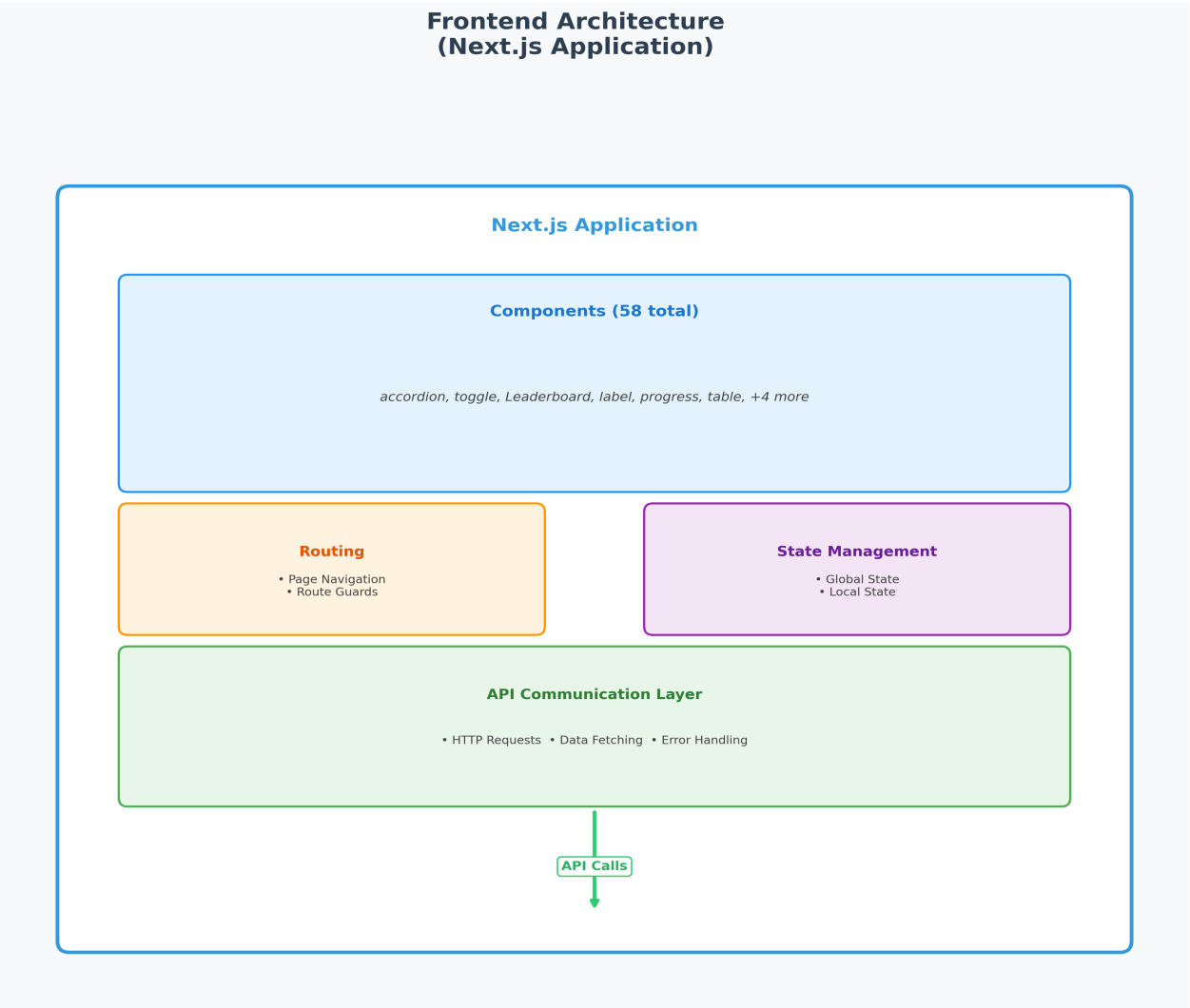
Framework & Structure:

Primary Framework: Next.js, Vite, React

Component Analysis:

Total Components: 58

Total Pages: 0



Component Structure:

- Main frontend folders: src\app\components\ui, src\app\components, root
- Total frontend files: 58
- Component-based architecture detected

5. Backend Architecture

Framework & Structure:

Primary Framework: Python, FastAPI, Uvicorn

API Endpoints (6 detected):

- GET /api/UX

Purpose: GET endpoint for UX

- GET /api/paginationx

Purpose: Retrieve paginationx records

- PUT /api/paginationx/{id}

Purpose: Update paginationx

- GET /api/gamehistoryx

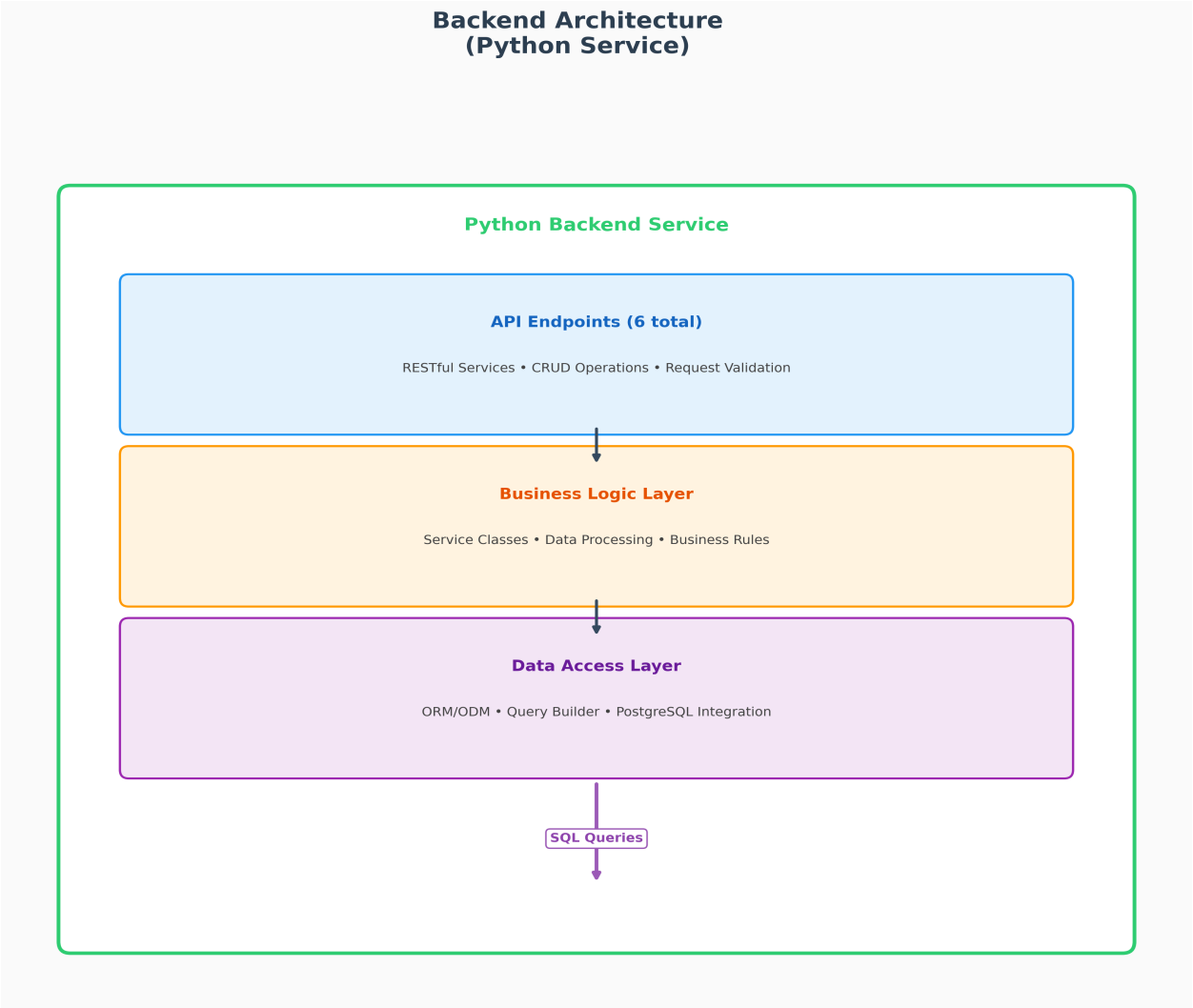
Purpose: Retrieve gamehistoryx records

- PUT /api/gamehistoryx/{id}

Purpose: Update gamehistoryx

- GET /api/sheetx

Purpose: Retrieve sheetx records



Service Layer Analysis:

- HTTP methods distribution: GET: 4, PUT: 2
- Service domains identified: Ux, Paginationx, Gamehistoryx, Sheetx
- CRUD operations supported: Update, Read

6. API Endpoints Documentation

Inferred API Endpoints (Based on Frontend & PRD Analysis):

GET /api/UX

Purpose: GET endpoint for UX

Response Fields:

Field	Type	Description
data	array	Array of UX objects
total	integer	Total count

GET /api/paginationx

Purpose: Retrieve paginationx records

Request Fields:

Field	Type	Required	Description
page	integer	No	Page number
limit	integer	No	Records per page

Response Fields:

Field	Type	Description
data	array	Array of paginationx objects
total	integer	Total count
name	string	Paginationx name
description	string	Paginationx description

POST /api/paginationx

Purpose: Create new paginationx

Request Fields:

Field	Type	Required	Description
name	string	Yes	Paginationx name
description	string	No	Paginationx description

Response Fields:

Field	Type	Description
id	integer	Created paginationx ID
message	string	Success message

PUT /api/paginationx/{id}

Purpose: Update paginationx

Request Fields:

Field	Type	Required	Description
name	string	Yes	Paginationx name
description	string	No	Paginationx description

Response Fields:

Field	Type	Description
message	string	Update confirmation

GET /api/gamehistoryx

Purpose: Retrieve gamehistoryx records

Request Fields:

Field	Type	Required	Description
page	integer	No	Page number
limit	integer	No	Records per page

Response Fields:

Field	Type	Description
data	array	Array of gamehistoryx objects
total	integer	Total count
name	string	Gamehistoryx name
description	string	Gamehistoryx description

POST /api/gamehistoryx

Purpose: Create new gamehistoryx

Request Fields:

Field	Type	Required	Description
name	string	Yes	Gamehistoryx name
description	string	No	Gamehistoryx description

Response Fields:

Field	Type	Description
id	integer	Created gamehistoryx ID
message	string	Success message

PUT /api/gamehistoryx/{id}

Purpose: Update gamehistoryx

Request Fields:

Field	Type	Required	Description
name	string	Yes	Gamehistoryx name
description	string	No	Gamehistoryx description

Response Fields:

Field	Type	Description
message	string	Update confirmation

GET /api/sheetx

Purpose: Retrieve sheetx records

Request Fields:

Field	Type	Required	Description
page	integer	No	Page number
limit	integer	No	Records per page

Response Fields:

Field	Type	Description
data	array	Array of sheetx objects
total	integer	Total count
name	string	Sheetx name
description	string	Sheetx description

POST /api/sheetx

Purpose: Create new sheetx

Request Fields:

Field	Type	Required	Description
name	string	Yes	Sheetx name
description	string	No	Sheetx description

Response Fields:

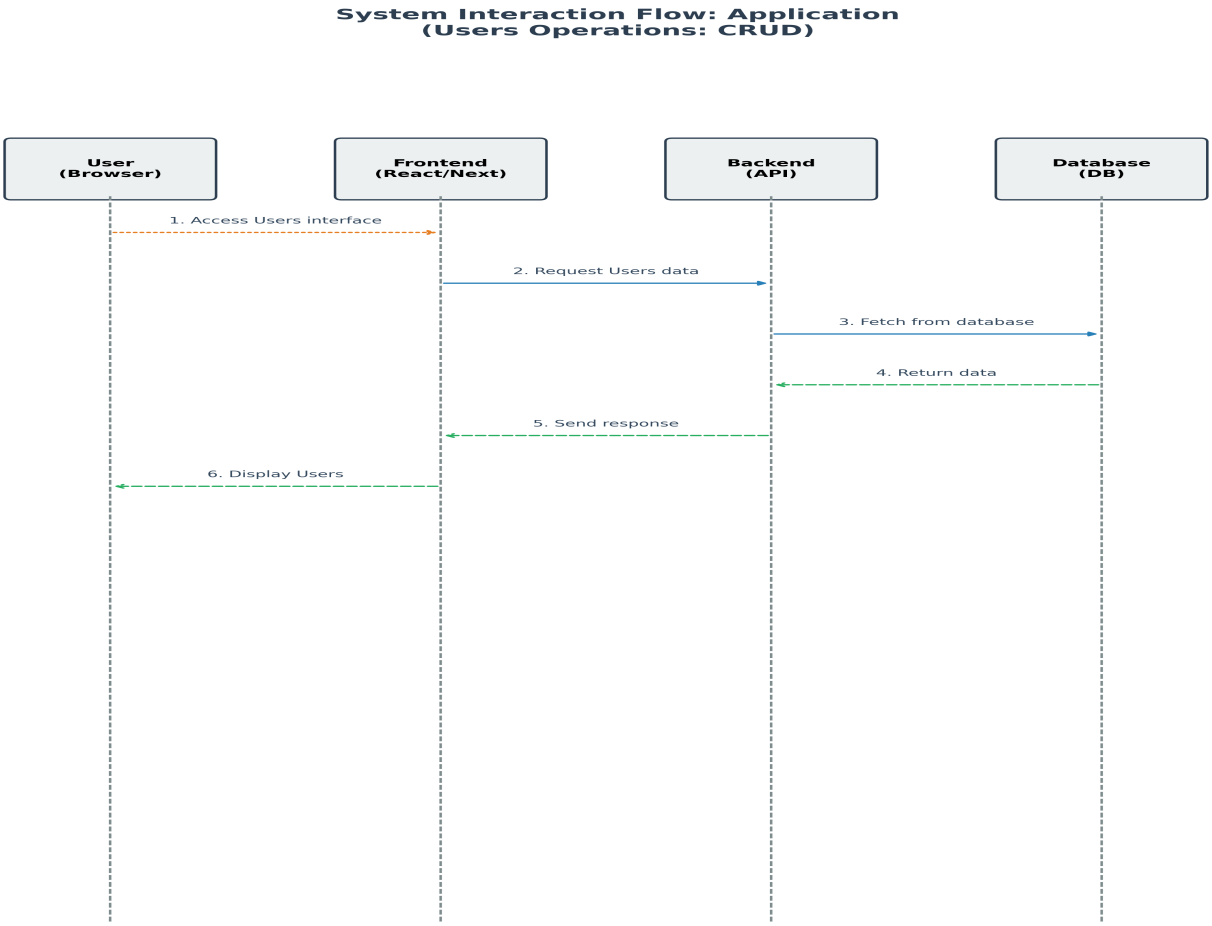
Field	Type	Description
id	integer	Created sheetx ID
message	string	Success message

Total API Endpoints: 9

7. System Interaction Flow (Sequence Diagram)

Interaction Scenario: Users Management (View & Create)

This sequence diagram illustrates the end-to-end flow for managing Users within the Application. It demonstrates the interaction between the User, Frontend, Backend, and Database layers.



Flow Description:

1. **User Action:** User navigates to the users page in the browser.
2. **Frontend Fetch:** The frontend initiates a GET request to /api/users.
3. **Database Query:** The backend executes a SELECT * FROM users query.
4. **Data Render:** The list of records is returned and displayed to the user.
5. **User Action:** User clicks 'Create' and submits the form.
6. **Creation Request:** Frontend sends a POST request to /api/users with payload.
7. **Persistence:** Backend executes INSERT INTO users to save the new record.
8. **Confirmation:** A success response (201 Created) is returned, and the UI updates.

8. Component Interactions

Communication Patterns:

- RESTful API communication
- JSON data exchange
- HTTP/HTTPS protocols

9. Database Details

Architecture Summary:

- Frontend: Next.js, Vite, React
- Backend: Python, FastAPI, Uvicorn
- Database: PostgreSQL, SQLAlchemy

Database Details:

- Primary Database: PostgreSQL - Relational database for structured data storage
- ORM Layer: SQLAlchemy - Python SQL toolkit and Object-Relational Mapping
- Connection Pooling: Configured for optimal performance and resource management
- Transaction Management: ACID compliance with rollback capabilities
- Security: Encrypted connections, parameterized queries to prevent SQL injection
- Backup Strategy: Automated daily backups with point-in-time recovery
- Indexing: Optimized indexes on frequently queried columns for performance

Database Tables (Based on Detected API Endpoints):

- Paginationx Table: Paginationx data storage and management
- - id (Primary Key, Auto-increment): Unique identifier
- - name (VARCHAR(255), NOT NULL): Paginationx name
- - description (VARCHAR(255)): Paginationx description
- - created_at (TIMESTAMP, NOT NULL, DEFAULT CURRENT_TIMESTAMP): Record creation time
- - updated_at (TIMESTAMP, NULL, ON UPDATE CURRENT_TIMESTAMP): Last modification time
- Indexes:
- - INDEX idx_paginationx_name (name)
- Gamehistoryx Table: Gamehistoryx data storage and management
- - id (Primary Key, Auto-increment): Unique identifier
- - name (VARCHAR(255), NOT NULL): Gamehistoryx name
- - description (VARCHAR(255)): Gamehistoryx description
- - created_at (TIMESTAMP, NOT NULL, DEFAULT CURRENT_TIMESTAMP): Record creation time
- - updated_at (TIMESTAMP, NULL, ON UPDATE CURRENT_TIMESTAMP): Last modification time
- Indexes:
- - INDEX idx_gamehistoryx_name (name)
- Sheetx Table: Sheetx data storage and management
- - id (Primary Key, Auto-increment): Unique identifier
- - name (VARCHAR(255), NOT NULL): Sheetx name
- - description (VARCHAR(255)): Sheetx description
- - created_at (TIMESTAMP, NOT NULL, DEFAULT CURRENT_TIMESTAMP): Record creation time
- - updated_at (TIMESTAMP, NULL, ON UPDATE CURRENT_TIMESTAMP): Last modification time
- Indexes:

- - INDEX idx_sheetx_name (name)
- Users Table: Users data storage and management
- - id (Primary Key, Auto-increment): Unique identifier
- - name (VARCHAR(255), NOT NULL): Users name
- - description (VARCHAR(255)): Users description
- - created_at (TIMESTAMP, NOT NULL, DEFAULT CURRENT_TIMESTAMP): Record creation time
- - updated_at (TIMESTAMP, NULL, ON UPDATE CURRENT_TIMESTAMP): Last modification time
- Indexes:
- - INDEX idx_users_name (name)
- S Table: S data storage and management
- - id (Primary Key, Auto-increment): Unique identifier
- - name (VARCHAR(255), NOT NULL): Sheetx name
- - description (VARCHAR(255)): Sheetx description
- - created_at (TIMESTAMP, NOT NULL, DEFAULT CURRENT_TIMESTAMP): Record creation time
- - updated_at (TIMESTAMP, NULL, ON UPDATE CURRENT_TIMESTAMP): Last modification time
- Indexes:
- - INDEX idx_s_name (name)

10. Deployment Architecture

Deployment Configuration:

- Containerization: Recommended
- Cloud Readiness: High
- Environment Configuration: Standard
- Monitoring Setup: Recommended

11. Security Model

Security Recommendations:

- Implement JWT token authentication
- Use HTTPS for all communications
- Validate and sanitize user inputs
- Implement rate limiting on APIs
- Regular security audits

12. Technology Stack Summary

Frontend Technologies:

- Next.js
- Vite
- React

Backend Technologies:

- Python
- FastAPI
- Uvicorn

Database Technologies:

- PostgreSQL
- SQLAlchemy

13. Business Alignment

14. Recommendations & Next Steps

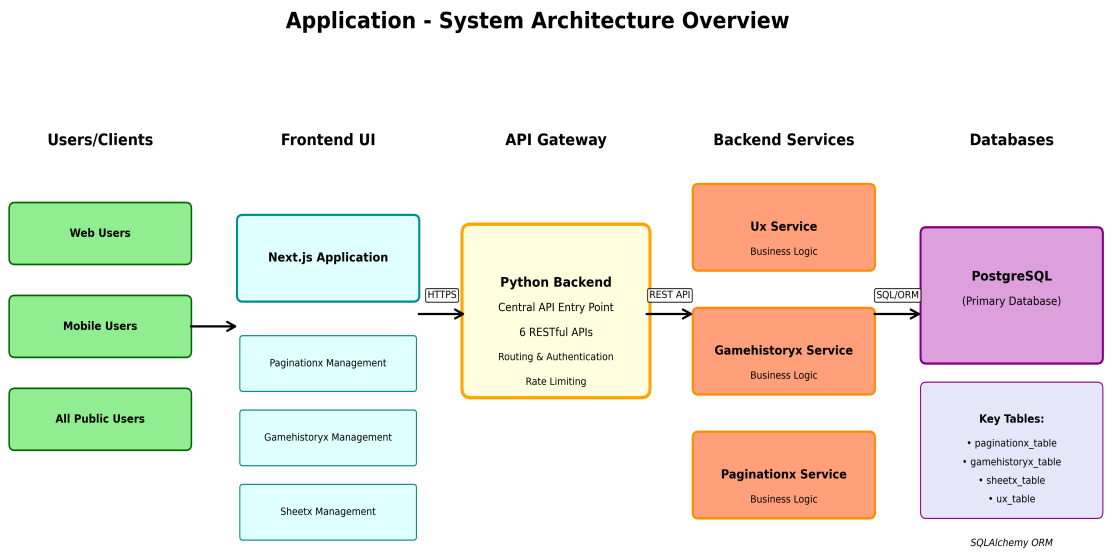
Architecture Recommendations:

1. Implement containerization with Docker
2. Add comprehensive testing framework
3. Implement monitoring and logging
4. Add error handling and validation
5. Consider implementing caching strategies
6. Regular security audits and updates

15. System Architecture Diagram

System Architecture Overview:

The following diagram shows the complete system architecture with real components, API endpoints, and data flow based on the analyzed repository and PRD.



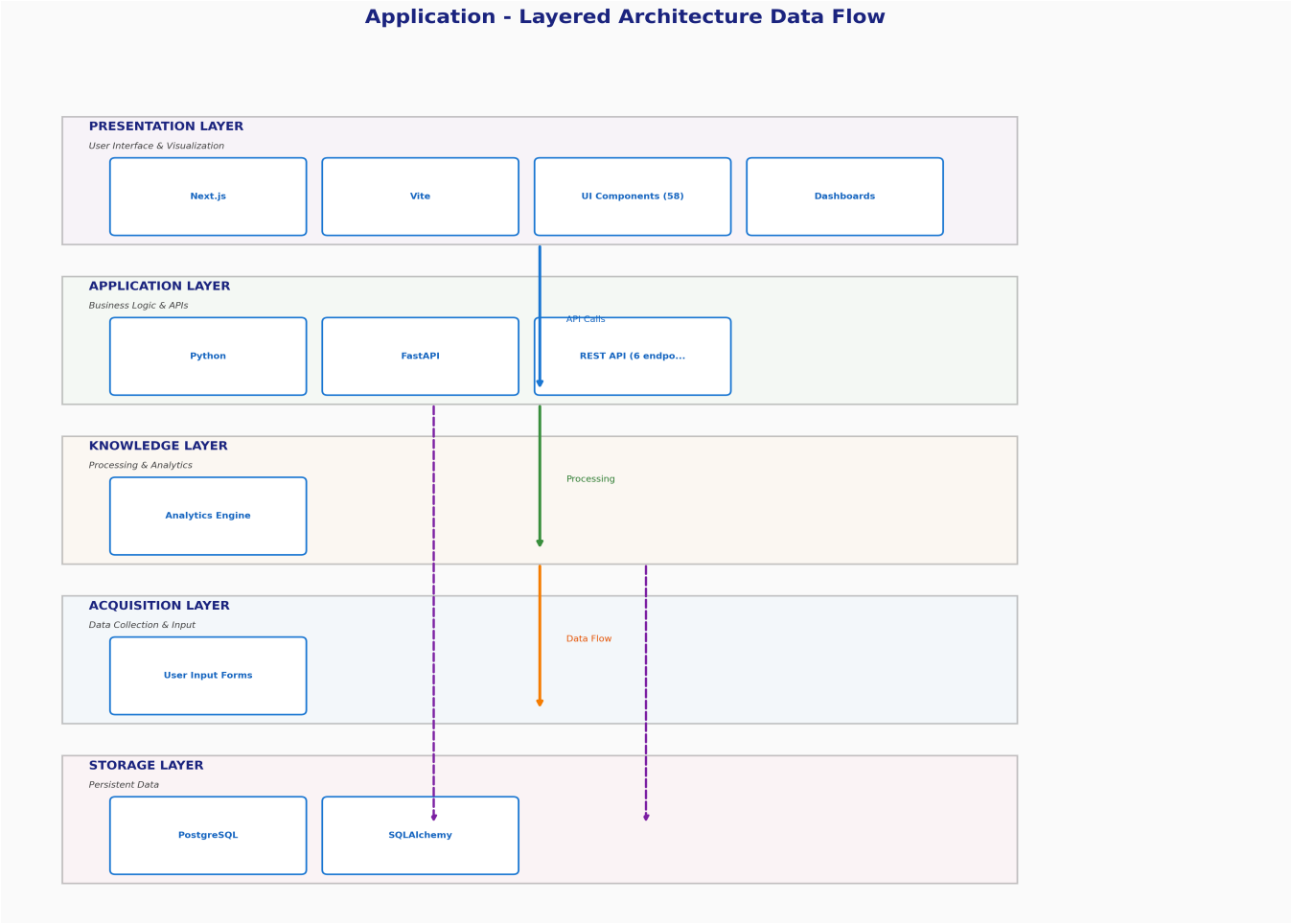
Architecture Insights:

- System manages 5 main entities through 6 API endpoints
- Full-stack architecture: Next.js frontend communicates with Python backend
- Data persistence handled by PostgreSQL with ORM integration

16. Layered Data Flow Architecture

Comprehensive Architecture Layers:

This diagram illustrates the complete layered architecture of Application, showing how data flows through different architectural layers from data acquisition to presentation. Each layer is dynamically detected from the repository analysis and PRD document.



Architecture Layers Explained:

- **Presentation Layer:** User interface components, dashboards, and visualization elements that users interact with directly.
- **Application Layer:** Business logic, REST APIs, and core application services that process requests and orchestrate operations.
- **Knowledge Layer:** Analytics, machine learning models, and data processing engines that derive insights from data.
- **Acquisition Layer:** Data collection mechanisms including user inputs, IoT sensors, batch processes, and external data sources.
- **Storage Layer:** Persistent data storage including databases, caches, and file storage systems.
- **Security Layer:** Authentication, authorization, encryption, and security controls protecting the system.

- **CI/CD Layer:** Continuous integration and deployment tools for building, testing, and deploying the application.
- **External Systems:** Third-party APIs and external services integrated with the application.